PYTHON 10 CASE STUDIES

Casestudy1:Retail Billing with Multiple Discount Rules: Process multiple products, quantities, category specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

Problem statement: Develop a Python program for a retail shop that accepts multiple products, their prices, and quantities purchased by a customer. Calculate the total price, apply a discount based on the purchase amount, calculate tax, and display the final bill.

Objective:

1. Accept product details from the customer.
2. Calculate the cost of each product.
3. Calculate the subtotal of all products.
4. Apply a discount based on the subtotal.
5. Calculate tax and the final payable amount.
6. Display an itemized bill.

Assumptions:

- The customer purchases at least one product.
- Product prices and quantities must be positive.
- A 10% discount applies when the subtotal is ₹2,000 or more.
- Otherwise, no discount is applied.
- GST is assumed to be 5% of the amount after discount.
- All amounts are calculated in Indian rupees.

Algorithm:

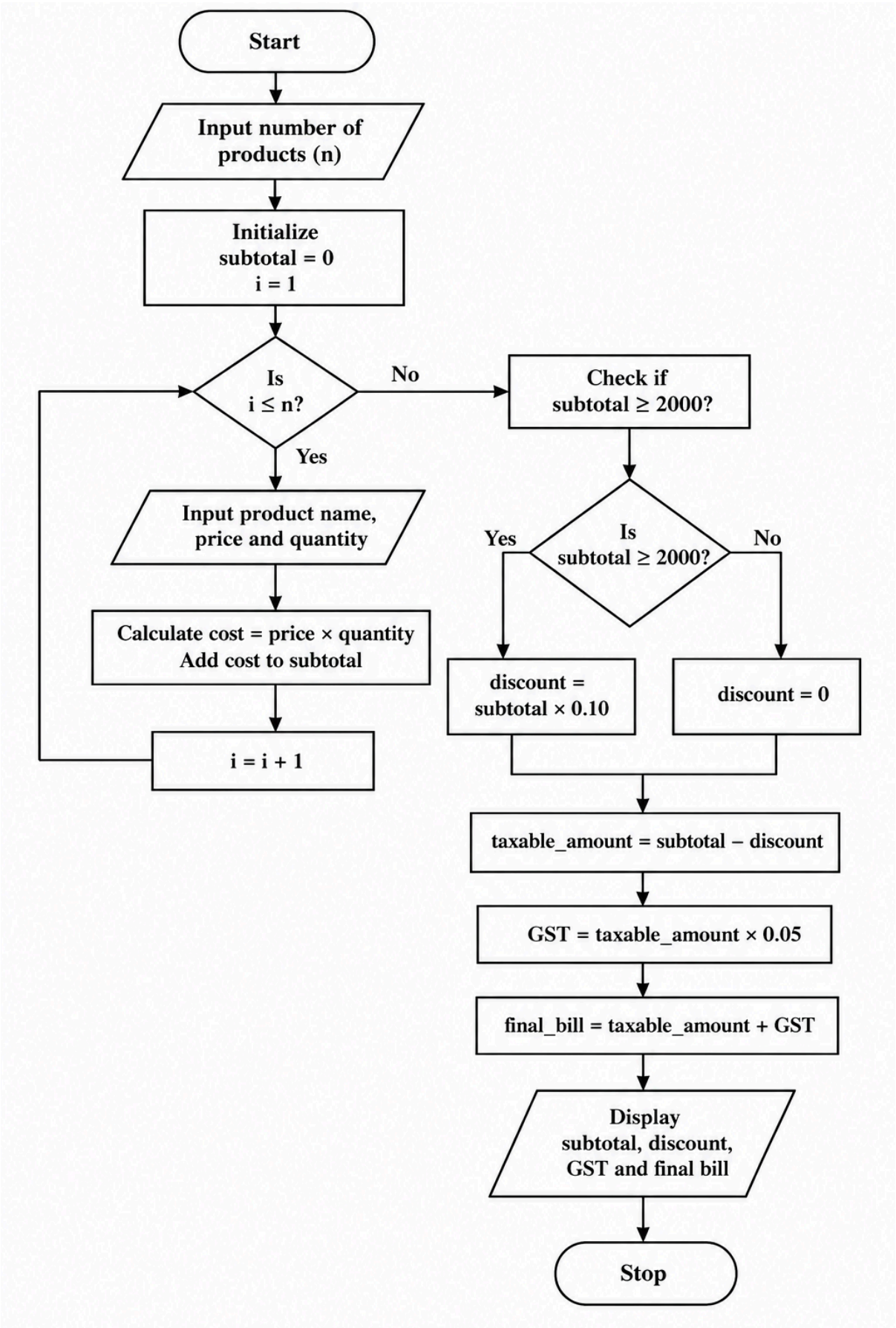
Start.

1. Input the number of products.
2. Initialize the subtotal to zero.
3. For each product:

- Input its name, price, and quantity.
- Calculate `price*quantity`.
- Add the product cost to the subtotal.

5. If the subtotal is at least ₹2,000, calculate a 10% discount; otherwise, set the discount to zero.
6. Calculate the taxable amount by subtracting the discount from the subtotal.
7. Calculate GST at 5%.
8. Calculate the final bill by adding GST to the taxable amount.
9. Display the subtotal, discount, GST, and final bill.
10. Stop.

Flowchart:


Start
Input number of products (n)
Initialize subtotal = 0 i = 1
Is i ≤ n?
No
Yes
Input product name, price and quantity
Calculate cost = price × quantity Add cost to subtotal
i = i + 1
Check if subtotal ≥ 2000?
Is subtotal ≥ 2000?
Yes
No
discount = subtotal × 0.10
discount = 0
taxable_amount = subtotal − discount
GST = taxable_amount × 0.05
final_bill = taxable_amount + GST
Display subtotal, discount, GST and final bill
Stop

Pseudocode:

```
START
input n
subtotal = 0
for each product from 1 to n:
    input name, price, quantity
    cost = price * quantity
    subtotal = subtotal + cost

if subtotal >= 2000:
    discount = subtotal * 0.10
else:
    discount = 0

taxable_amount = subtotal - discount
GST = taxable_amount * 0.05
final_bill = taxable_amount + GST

Display subtotal
Display discount
Display GST
Display final_bill
STOP
```

Python source code:

```
n = int(input("Enter number of products: "))
subtotal = 0
for i in range(1, n + 1):
    print("Product", i)
    name = input("Enter product name: ")
    price = float(input("Enter price: "))
    quantity = int(input("Enter quantity: ")
```

```
    cost = price * quantity
    subtotal += cost
    print(name, "Cost =", cost)
if subtotal >= 2000:
    discount = subtotal * 0.10
else:
    discount = 0
taxable_amount = subtotal - discount
gst = taxable_amount * 0.05
final_bill = taxable_amount + gst
print("----- FINAL BILL -----")
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("GST: Rs.", round(gst, 2))
print("Final Bill: Rs.", round(final_bill, 2))
```

Testcases:

#1

```
Enter number of products: 1

Product 1
Enter product name: Pen
Enter price: 500
Enter quantity: 2
Pen Cost = 1000.0


Subtotal: Rs. 1000.0
Discount: Rs. 0
GST: Rs. 50.0
Final Bill: Rs. 1050.0
```

#2

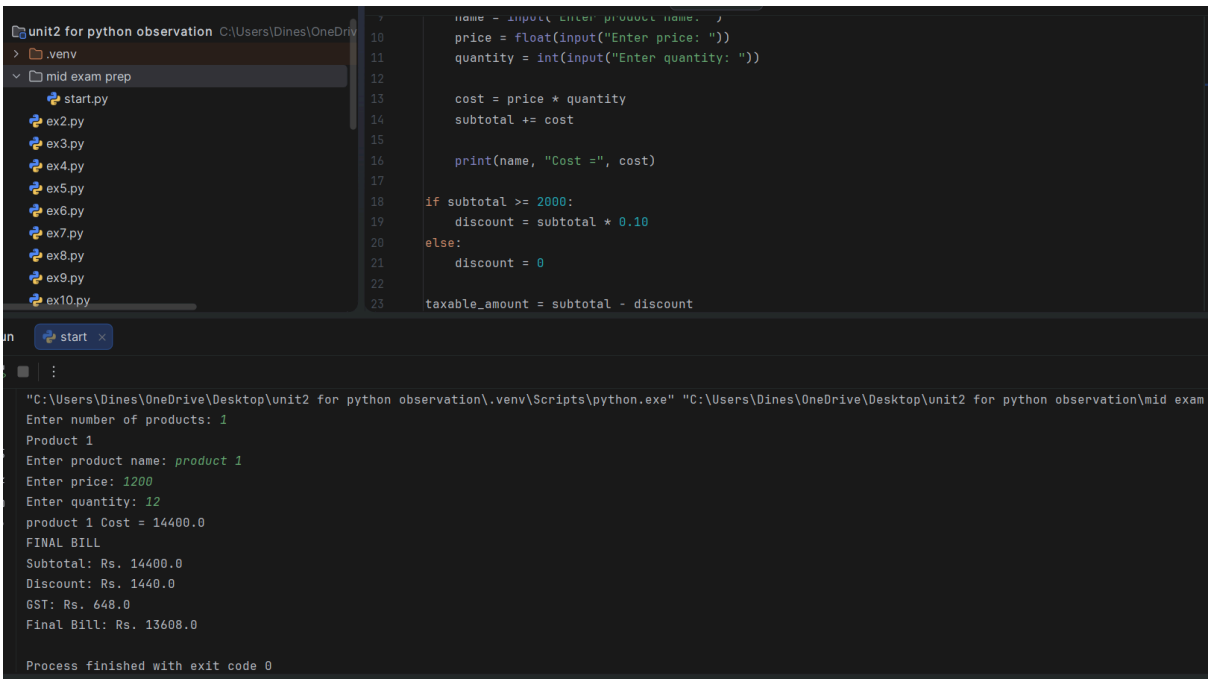
```
Enter number of products: 2

Product 1
Enter product name: Bag
Enter price: 1200
Enter quantity: 1
Bag Cost = 1200.0

Product 2
Enter product name: Book
Enter price: 1000
Enter quantity: 1
Book Cost = 1000.0

Subtotal: Rs. 2200.0
Discount: Rs. 220.0
GST: Rs. 99.0
Final Bill: Rs. 2079.0
```

Output Screenshots:



Conclusion:

The Retail Billing System demonstrates how Python can be used to automate billing calculations. It calculates product costs, applies conditional discounts, computes GST, and generates the final payable amount. This case study helps students understand arithmetic operations, conditional statements, loops, and basic input/output.

Casestudy2:Examination Result and Grace-Mark Validation: Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

Problem Statement:
Develop a Python program to calculate a student's examination result based on marks obtained in multiple subjects. The program should check pass marks, apply grace marks according to specified rules, and display the final result and total marks.

Objectives:

1. Accept marks for multiple subjects.
2. Validate the marks entered by the user.
3. Check whether the student passes each subject.
4. Apply grace marks when the specified conditions are satisfied.
5. Calculate total marks, percentage, and final result.

Assumptions:

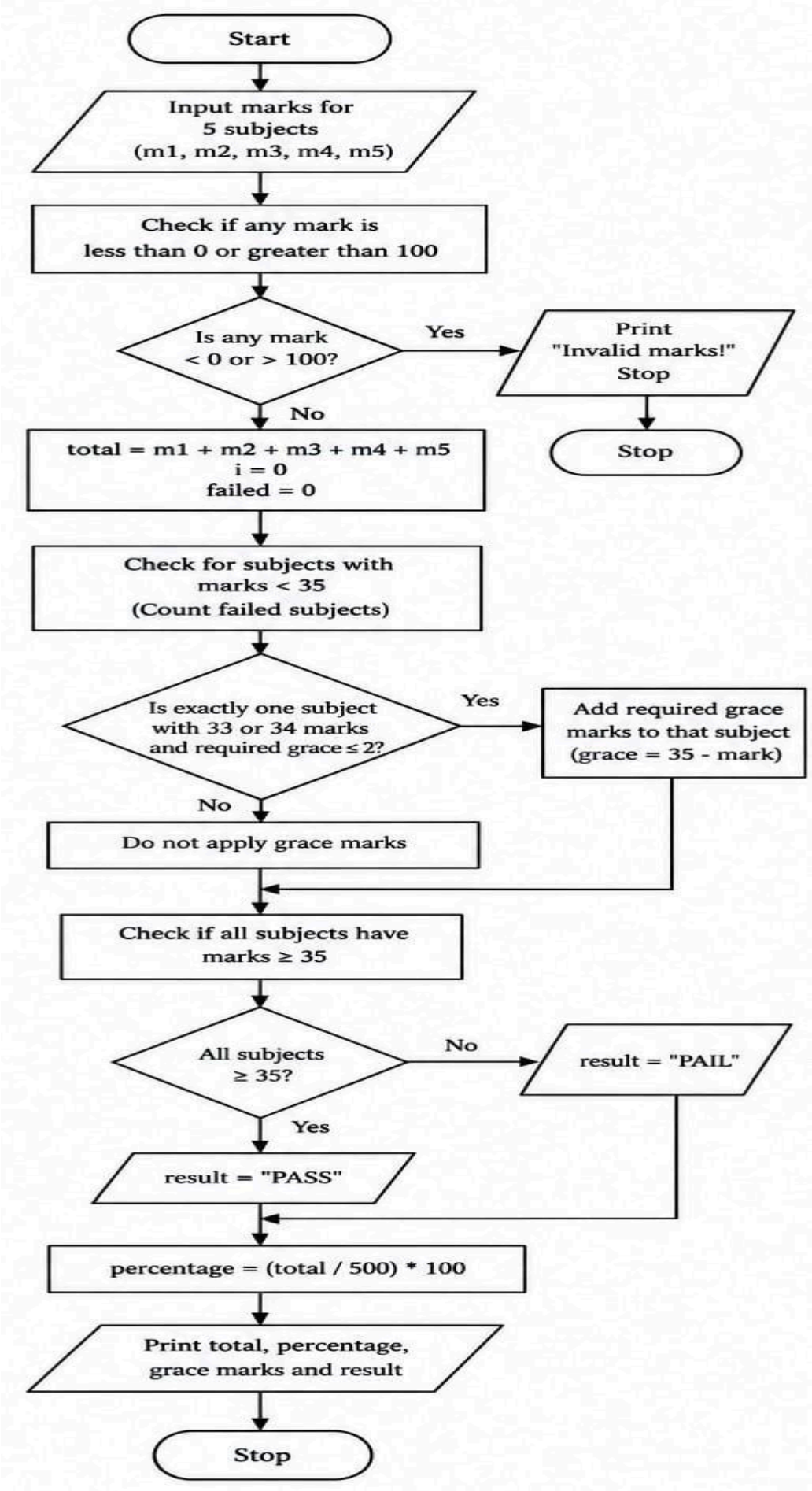
- The student has five subjects.
- Each subject carries a maximum of 100 marks.
- The minimum passing mark is 35 in each subject.
- A student who scores 33 or 34 in exactly one subject can receive enough grace marks to reach 35.
- Grace marks are limited to a maximum of 2 marks in total.
- Marks must be between 0 and 100.

Algorithm:

1. Start.

2. Initialize the total marks to zero.
3. Input marks for five subjects.
4. Check whether each mark is between 0 and 100.
5. Calculate the total marks.
6. Count the subjects in which the student scored below 35.
7. If exactly one subject has 33 or 34 marks and the required grace marks do not exceed 2, apply grace marks.
8. Check whether the student passes all subjects after applying grace marks.
9. Calculate the percentage.
10. Display the total, percentage, grace marks awarded,and result.
11. Stop.

Flowchart:


Start
Input marks for 5 subjects (m1, m2, m3, m4, m5)
Check if any mark is less than 0 or greater than 100
Is any mark < 0 or > 100?
Yes
Print "Invalid marks!" Stop
Stop
No
total = m1 + m2 + m3 + m4 + m5
i = 0
failed = 0
Check for subjects with marks < 35 (Count failed subjects)
Is exactly one subject with 33 or 34 marks and required grace ≤ 2?
Yes
Add required grace marks to that subject (grace = 35 - mark)
No
Do not apply grace marks
Check if all subjects have marks ≥ 35
All subjects ≥ 35?
No
result = "PAIL"
Yes
result = "PASS"
percentage = (total / 500) * 100
Print total, percentage, grace marks and result
Stop

Pseudocode:

START

input m1, m2, m3, m4, m5

total = m1 + m2 + m3 + m4 + m5

grace = 0

if m1 is below 35:

if m1 is 33 or 34 and all other marks are at least 35:

grace = 35 - m1

m1 = 35

if m2 is below 35:

if m2 is 33 or 34 and all other marks are at least 35:

grace = 35 - m2

m2 = 35

if m3 is below 35:

if m3 is 33 or 34 andall other marks are at least 35:

grace = 35 - m3

m3 = 35

if m4 is below 35:

if m4 is 33 or 34 and all other marks are at least 35:

grace = 35 - m4

m4 = 35

if m5 is below 35:

  if m5 is 33 or 34 and all other marks are at least 35:

        grace = 35 - m5

        m5 = 35

total = m1 + m2 + m3 + m4 + m5

percentage = total / 5

Display total

Display percentage

Display grace

 if all marks are at least 35:

    Display "PASS"

else:

Display "FAIL"

STOP

Python source code:

```
m1 = int(input("Enter marks 1: "))
m2 = int(input("Enter marks 2: "))
m3 = int(input("Enter marks 3: "))
m4 = int(input("Enter marks 4: "))
m5 = int(input("Enter marks 5: "))
```

```
total = m1 + m2 + m3 + m4 + m5
grace = 0

if m1 < 35:
    if m1 >= 33 and m2 >= 35 and m3 >= 35 and m4 >= 35 and
m5 >= 35:
        grace = 35 - m1
        m1 = 35

if m2 < 35:
    if m2 >= 33 and m1 >= 35 and m3 >= 35 and m4 >= 35 and
m5 >= 35:
        grace = 35 - m2
        m2 = 35

if m3 < 35:
    if m3 >= 33 and m1 >= 35 and m2 >= 35 and m4 >= 35 and
m5 >= 35:
        grace = 35 - m3
        m3 = 35

if m4 < 35:
    if m4 >= 33 and m1 >= 35 and m2 >= 35 and m3 >= 35 and
m5 >= 35:
        grace = 35 - m4
        m4 = 35
```

```
if m5 < 35:
    if m5 >= 33 and m1 >= 35 and m2 >= 35 and m3 >= 35 and
m4 >= 35:
        grace = 35 - m5
        m5 = 35

total = m1 + m2 + m3 + m4 + m5
percentage = total / 5

print("Total Marks:", total)
print("Percentage:", percentage, "%")
print("Grace Marks:", grace)

if m1 >= 35 and m2 >= 35 and m3 >= 35 and m4 >= 35 and m5
>= 35:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

Testcases:

#1

Enter marks for Subject 1: 80

Enter marks for Subject 2: 75

Enter marks for Subject 3: 68

Enter marks for Subject 4: 90

Enter marks for Subject 5: 85

Total Marks: 398

Percentage: 79.6 %

Grace Marks Awarded: 0

Result: pass

#2

Enter marks for Subject 1: 80

Enter marks for Subject 2: 75

Enter marks for Subject 3: 34

Enter marks for Subject 4: 90

Enter marks for Subject 5: 85

Total Marks: 350

Percentage: 70.0 %

Grace Marks Awarded: 1

Result: PASS

Output Screenshots:



Conclusion:

The Examination Result and Grace-Mark Validation program demonstrates how Python can calculate total marks and percentages, validate input, and determine a student's result using conditional statements and lists. It also illustrates how grace-mark rules can be applied to examination results.

Casestudy3: **Attendance Recovery Computation: Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.**

Problem Statement:A college wants to check whether a student has enough attendance to attend examinations. The program takes the total number of classes conducted and the number of classes attended by the student. It

calculates the attendance percentage and checks whether the student meets the minimum attendance requirement.

Objectives:

1.Calculate the student's attendance percentage.

2.Check whether the student meets the minimum attendance requirement.

3.Display how many additional classes the student must attend to reach the requirement.

4.Display the final result clearly.

Assumptions

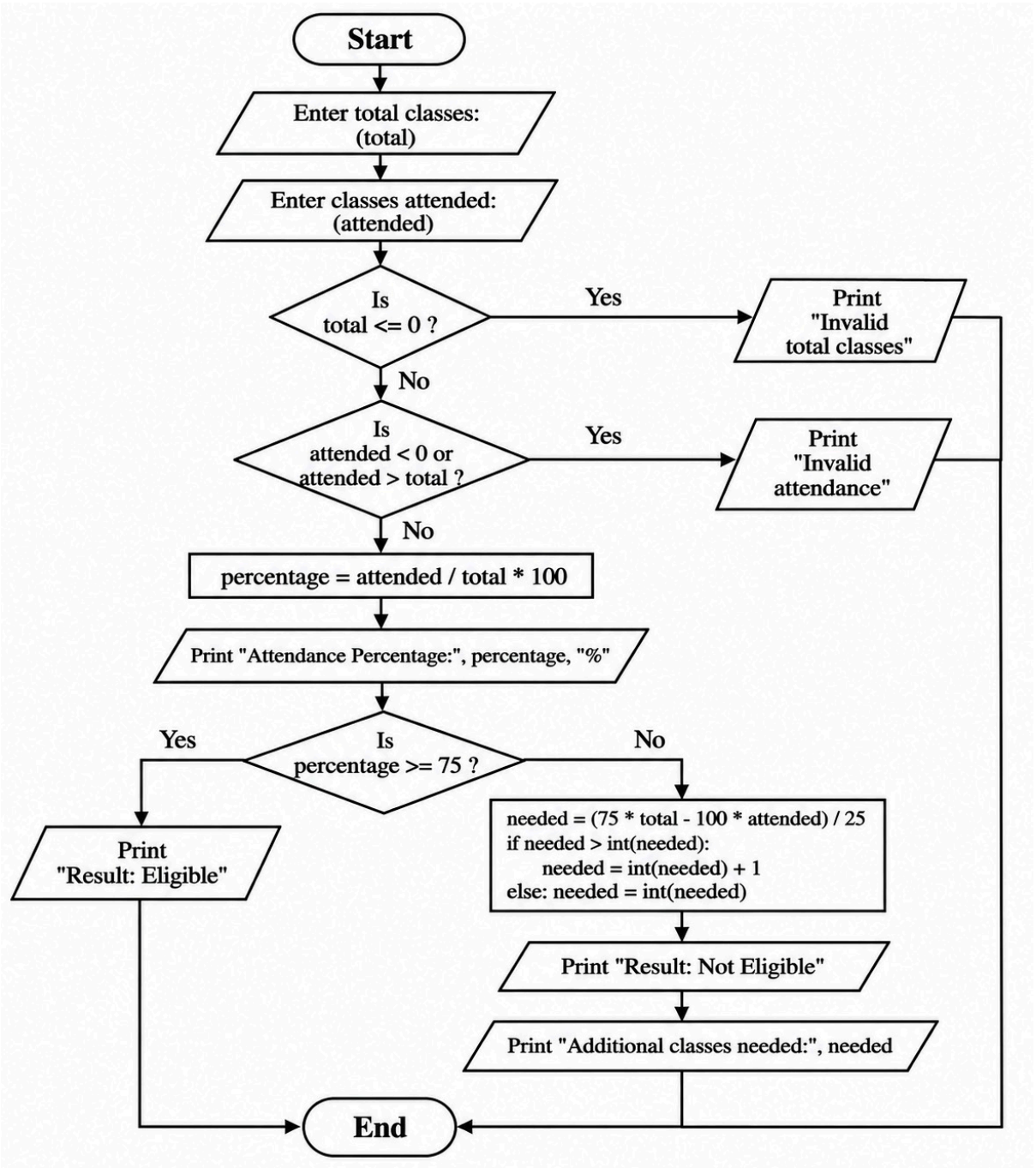
- Minimum required attendance is 75%.
- Total classes must be greater than zero.
- Attended classes cannot be greater than total classes.
- The student attends every upcoming class until reaching 75%.

Algorithm:

1. Start.

2. Enter the total number of classes.
3. Enter the number of classes attended.
4. Calculate attendance percentage.
5. If attendance is 75% or more, display “Eligible”.
6. Otherwise, calculate the number of additional classes needed.
7. Display the attendance percentage and result.
8. Stop.

Flowchart:



Pseudocode:

START

input total

```
input attended

if total <= 0:
Display "Invalid total classes"
else if attended < 0 or attended > total:
   Display"Invalid attendance"

else :
    percentage = attended / total * 100
 Display percentage

  if  percentage >= 75:
  Display "Eligible"

 else :
        needed = (75 * total - 100 * attended) / 25

       if  needed is not a whole number:
            needed = whole number + 1

        Display "Not Eligible"
```

Display needed

STOP

Python source code:

```
total = int(input("Enter total classes: "))
attended = int(input("Enter classes attended: "))
if total <= 0:
    print("Invalid total classes")
elif attended < 0 or attended > total:
    print("Invalid attendance")
else:
    percentage = attended / total * 100
    print("Attendance Percentage:", percentage, "%")
    if percentage >= 75:
        print("Result: Eligible")
    else:
        needed = 75 * total - 100 * attended
        needed = needed / 25
        if needed > int(needed):
```

```
        needed = int(needed) + 1
    else:
        needed = int(needed)
    print("Result: Not Eligible")
    print("Additional classes needed:", needed)
```

Testcases:

#1

Enter total classes: 100

Enter classes attended: 80

Attendance Percentage: 80.0 %

Result: Eligible

#2

Enter total classes: 100

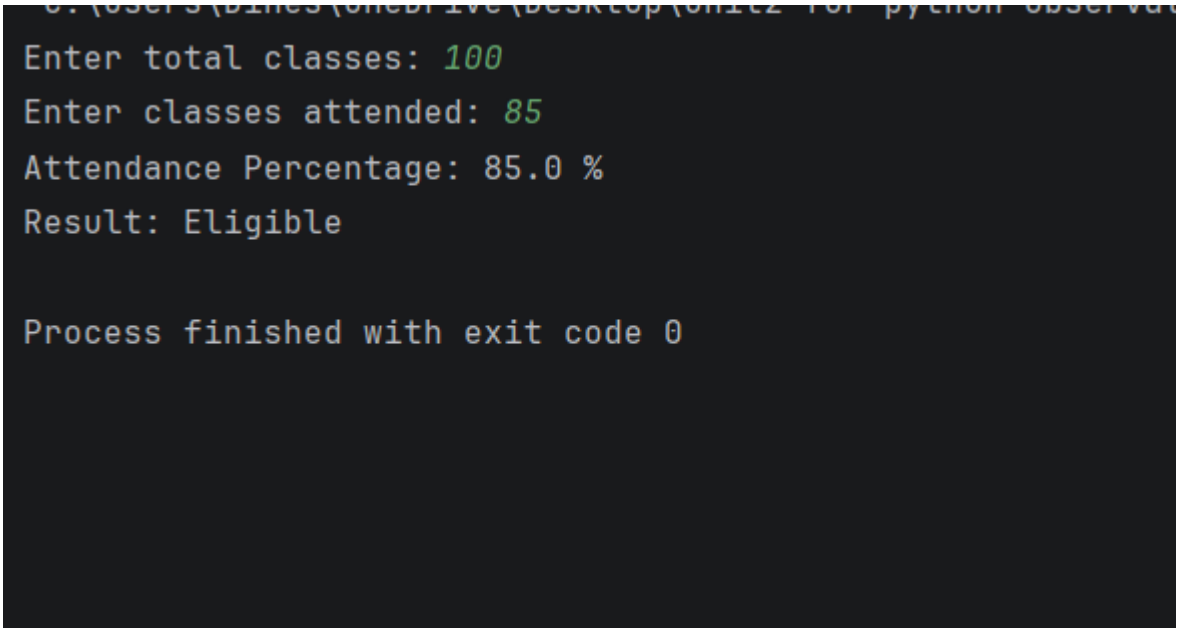
Enter classes attended: 60

Attendance Percentage: 60.0 %

Result: Not Eligible

Additional classes needed: 60

Output Screenshots:



Conclusion:

The Attendance Recovery Computation program calculates attendance percentage and checks examination eligibility. If attendance is below 75%, it calculates how many consecutive upcoming classes the student must attend to reach the required percentage.

Casestudy4:Password Policy Validation Engine: Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

Problem Statement:

Create a Python program to check whether a password follows basic password rules. The program checks the password length and verifies whether it contains an uppercase letter, a lowercase letter, a digit, and a special character.

Objectives:

1. Check whether the password meets the minimum length.
2. Check for uppercase and lowercase letters.
3. Check for at least one number.
4. Check for at least one special character.
5. Display whether the password is valid or invalid.

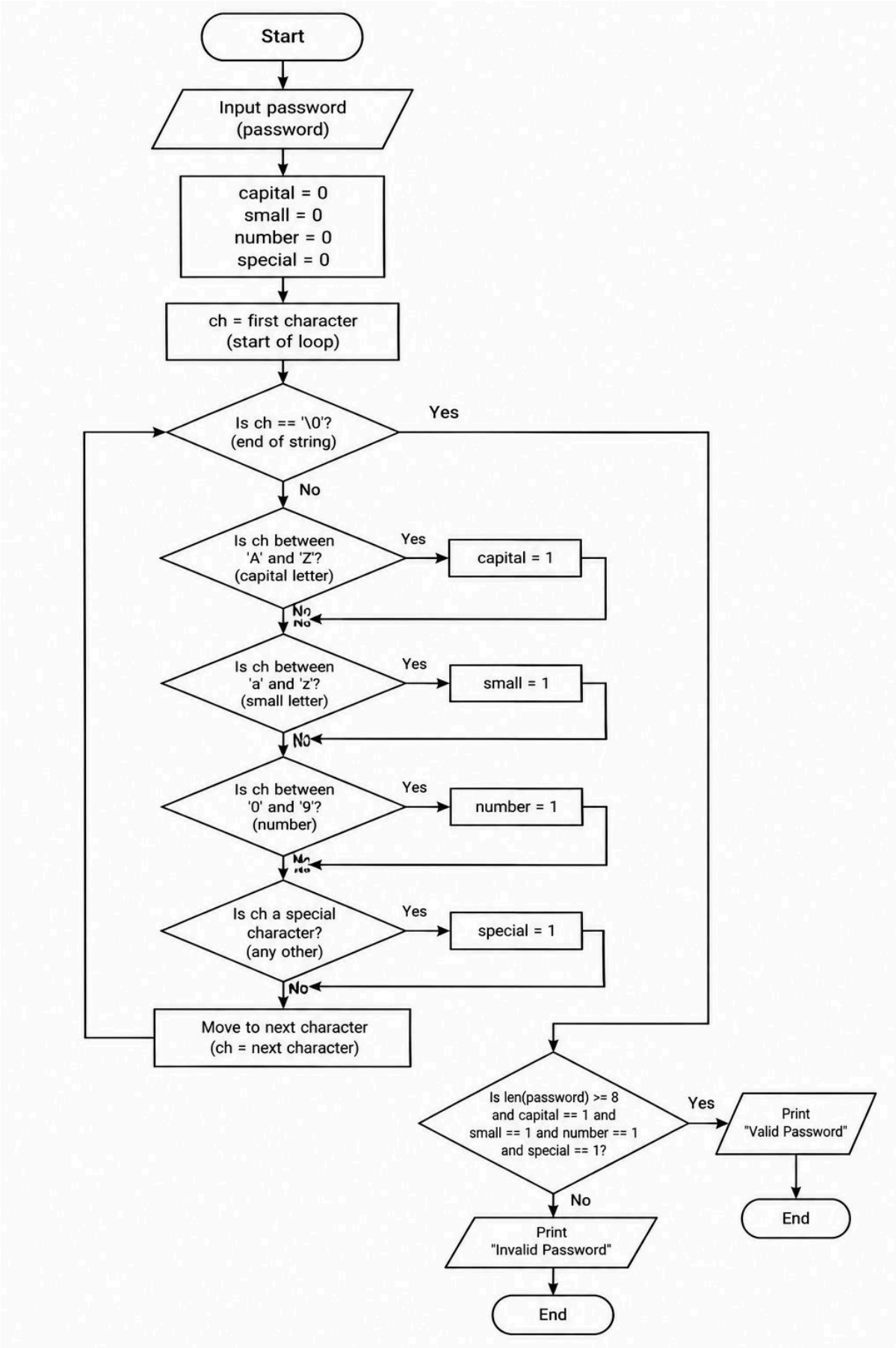
Assumptions:

- The password must contain at least 8 characters.
- It must contain at least one uppercase letter.
- It must contain at least one lowercase letter.
- It must contain at least one digit.
- It must contain at least one special character, such as @, #, or !.

Algorithm:

1. Start.
2. Enter a password.
3. Check whether its length is at least 8 characters.
4. Check for uppercase and lowercase letters.
5. Check for a digit and a special character.
6. If all conditions are satisfied, display “Valid Password”.
7. Otherwise, display “Invalid Password”.
8. Stop.

Flowchart:


Start
Input password (password)
capital = 0
small = 0
number = 0
special = 0
ch = first character (start of loop)
Is ch == '\0'? (end of string)
Yes
No
Is ch between 'A' and 'Z'? (capital letter)
Yes
capital = 1
No
Is ch between 'a' and 'z'? (small letter)
Yes
small = 1
No
Is ch between '0' and '9'? (number)
Yes
number = 1
No
Is ch a special character? (any other)
Yes
special = 1
No
Move to next character (ch = next character)
Is len(password) >= 8 and capital == 1 and small == 1 and number == 1 and special == 1?
Yes
Print "Valid Password"
End
No
Print "Invalid Password"
End

Pseudocode:

```
START
input password
Set capital = 0
Set small = 0
Set number = 0
Set special = 0
for each character in password
   if character is uppercase:
        Set capital = 1
 else if character is lowercase:
        Set small = 1
    else if character is a digit:
        Set number = 1
   else:
        Set special = 1
end for
if password length >= 8 and
   capital = 1 and small = 1 and
   number = 1 and special = 1:
```

```
    Display "Valid Password"
else:
    Display "Invalid Password"
STOP
```

Python source code:

```
password = input("Enter password: ")
capital = 0
small = 0
number = 0
special = 0
for ch in password:
    if ch >= 'A' and ch <= 'Z':
        capital = 1
    elif ch >= 'a' and ch <= 'z':
        small = 1
    elif ch >= '0' and ch <= '9':
        number = 1
    else:
        special = 1
```

```
if len(password) >= 8 and capital == 1 and small == 1 and
number == 1 and special == 1:
    print("Valid Password")
else:
    print("Invalid Password")
```

Testcases:

#1

Enter password: Hello@123

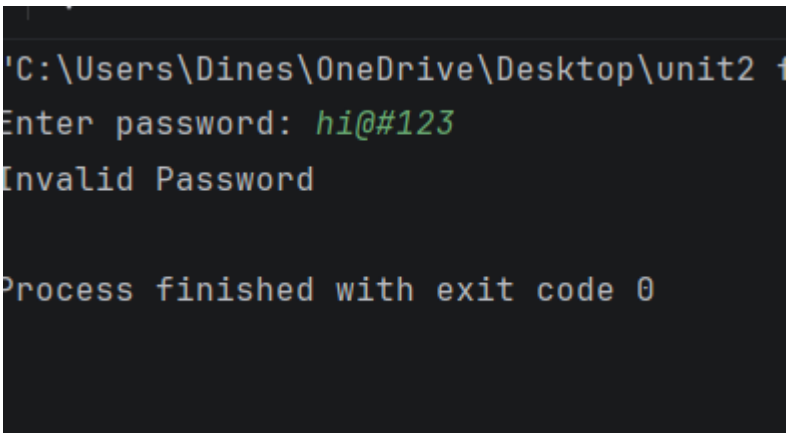
Valid Password

#2

Enter password: hello123

Invalid Password

Output Screenshots:



Conclusion:

The Password Policy Validation Engine checks whether a password follows the specified rules. It helps users create stronger passwords by requiring a combination of uppercase letters, lowercase letters, numbers, and special characters.

Casestudy5:Warehouse Inventory Reconciliation: Compare opening stock, incoming quantities, dispatched quantities, and closing stock across multiple products; detect discrepancies and calculate replenishment requirements.

Problem Statement:

A warehouse stores products and maintains their stock quantities. Sometimes, the actual stock may be different from the stock recorded in the system. This program compares the recorded stock with the actual stock and displays whether the stock matches.

Objectives:

1.Enter the recorded stock quantity.

2.Enter the actual stock quantity.

3.Compare both quantities.

4.Display whether the stock matches or has a difference.

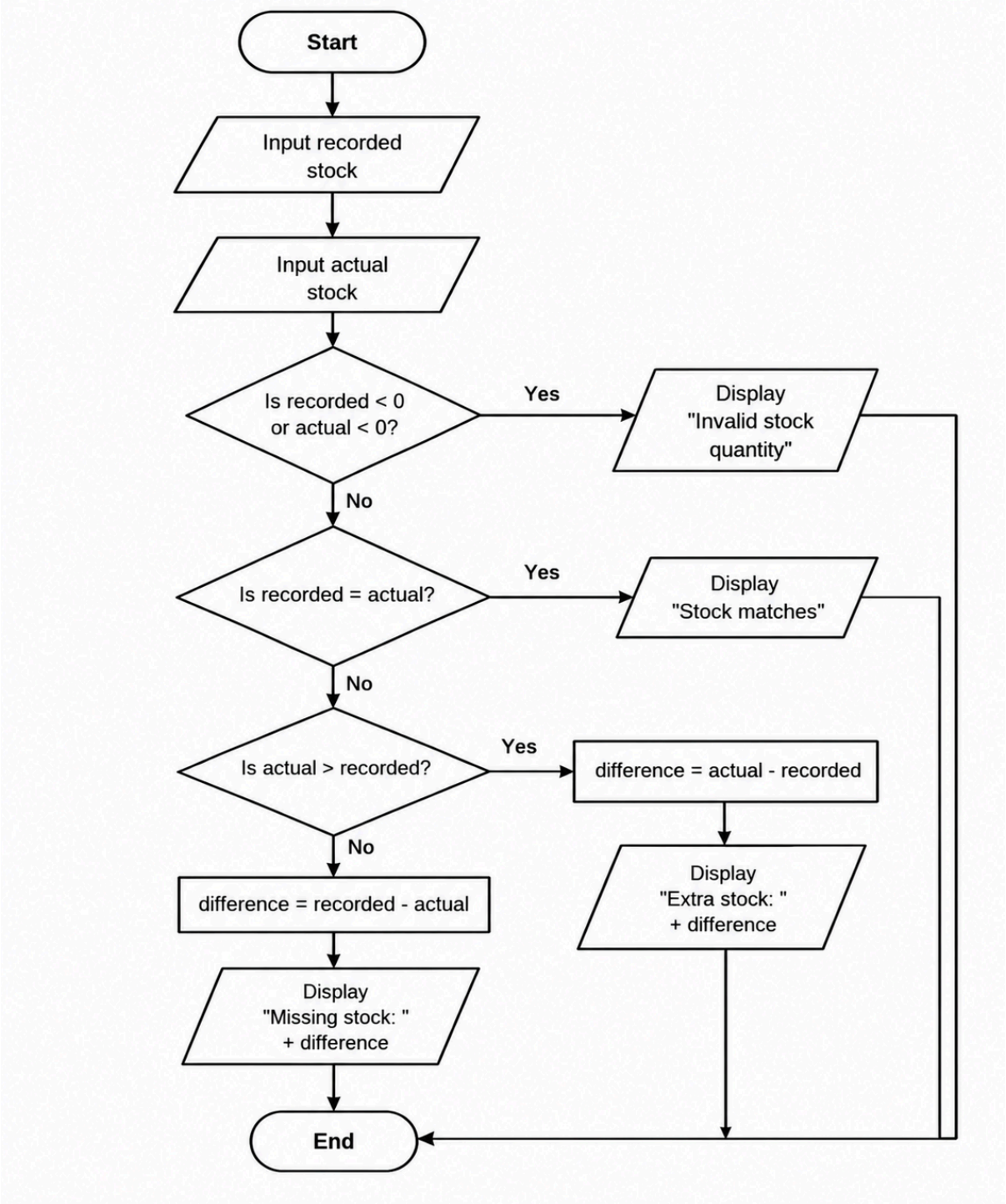
Assumptions:

- Stock quantities cannot be negative.
- Both quantities are whole numbers.
- If both quantities are equal, the stock matches.

Algorithm:

1. Start.
2. Enter the recorded stock.
3. Enter the actual stock.
4. Check whether either quantity is negative.
5. If valid, compare both quantities.
6. Display whether the stock matches or differs.
7. Stop.

Flowchart:


Start
Input recorded stock
Input actual stock
Is recorded < 0 or actual < 0?
Yes
Display "Invalid stock quantity"
No
Is recorded = actual?
Yes
Display "Stock matches"
No
Is actual > recorded?
Yes
difference = actual - recorded
Display "Extra stock: " + difference
No
difference = recorded - actual
Display "Missing stock: " + difference
End

Pseudocode:

```
 START
Input recorded
Input actual
if recorded < 0 or actual < 0:
    Display "Invalid stock quantity"
else if recorded == actual:
    Display "Stock matches"
else if actual > recorded:
    difference = actual - recorded
    Display "Extra stock:", difference
else:
    difference = recorded - actual
    Display "Missing stock:", difference

STOP
```

Python source code:

```
recorded = int(input("Enter recorded stock: "))
actual = int(input("Enter actual stock: "))
```

```
if recorded < 0 or actual < 0:
    print("Invalid stock quantity")
elif recorded == actual:
    print("Stock matches")
elif actual > recorded:
    difference = actual - recorded
    print("Extra stock:", difference)
else:
    difference = recorded - actual
    print("Missing stock:", difference)
```

Testcases:

#1

Enter recorded stock: 100

Enter actual stock: 100

Stock matches

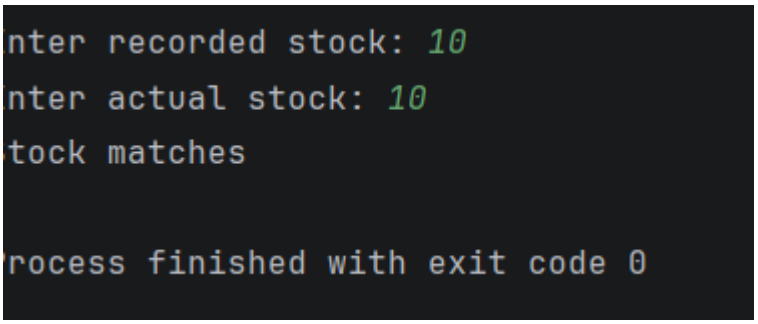
#2

Enter recorded stock: 100

Enter actual stock: 85

Missing stock: 15

Output Screenshots:



Conclusion:

The Warehouse Inventory Reconciliation program compares the recorded stock with the actual stock. It identifies matching stock, extra stock, or missing stock, helping warehouse staff find inventory differences.

Casestudy6:Smart Canteen Order and Stock Management: Process successive customer orders, calculate bills, update available inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

Problem Statement:

A college canteen sells food items to students. The program checks whether the requested quantity of a food item is available in stock. If enough stock is available, it calculates the

bill and updates the remaining stock. Otherwise, it displays an insufficient-stock message.

Objectives:

1. Enter the available stock quantity.
2. Enter the food item's price.
3. Enter the quantity ordered.
4. Check stock availability.
5. Calculate the total bill.
6. Display the remaining stock.

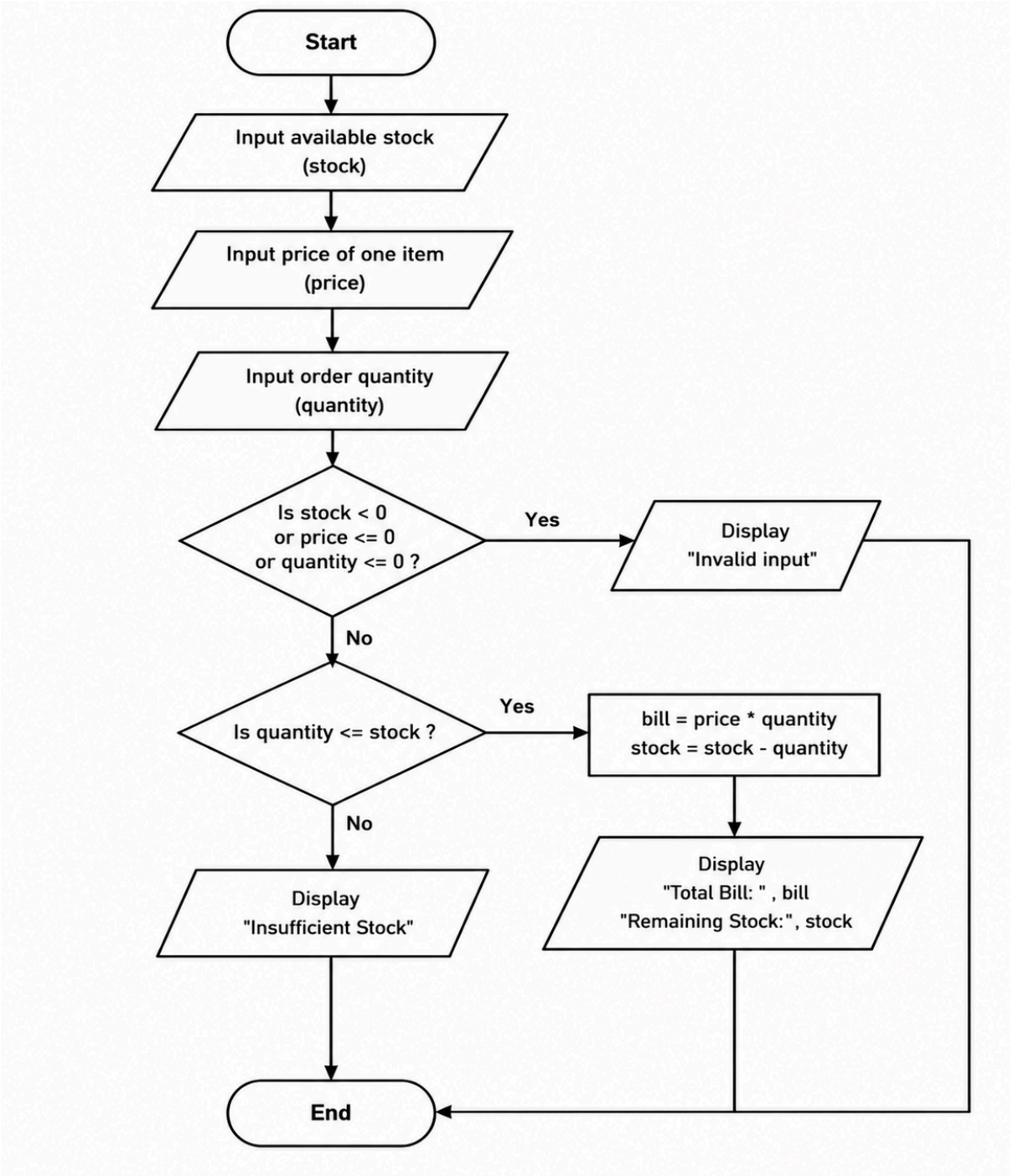
Assumptions:

- The food item's price must be greater than zero.
- Stock and order quantities cannot be negative.
- The order quantity must be greater than zero.
- The canteen sells only one type of food item in this simple program.

Algorithm:

1. Start.
2. Enter available stock.
3. Enter the price of one item.
4. Enter the ordered quantity.
5. Check whether the input values are valid.
6. Check whether the requested quantity is available.
7. If available, calculate the bill and remaining stock.
8. Otherwise, display “Insufficient Stock”.
9. Stop.

Flowchart:



Pseudocode:

START

Input stock

```
Input  price
Input  quantity
 if stock < 0 OR price <= 0 OR quantity <= 0:
    Display  "Invalid input"
elseif  quantity <= stock:
    bill = price * quantity
    stock = stock - quantity
  Display  bill
  Display  stock
else:
   Display  "Insufficient Stock"
STOP
```

Python source code:

```
stock = int(input("Enter available stock: "))
price = float(input("Enter price of one item: "))
quantity = int(input("Enter order quantity: "))
if stock < 0 or price <= 0 or quantity <= 0:
    print("Invalid input")
```

```
elif quantity <= stock:
    bill = price * quantity
    stock = stock - quantity
    print("Total Bill:", bill)
    print("Remaining Stock:", stock)
else:
    print("Insufficient Stock")
```

Testcases:

#1

Enter available stock: 50

Enter price of one item: 20

Enter order quantity: 3

Total Bill: 60.0

Remaining Stock: 47

#2

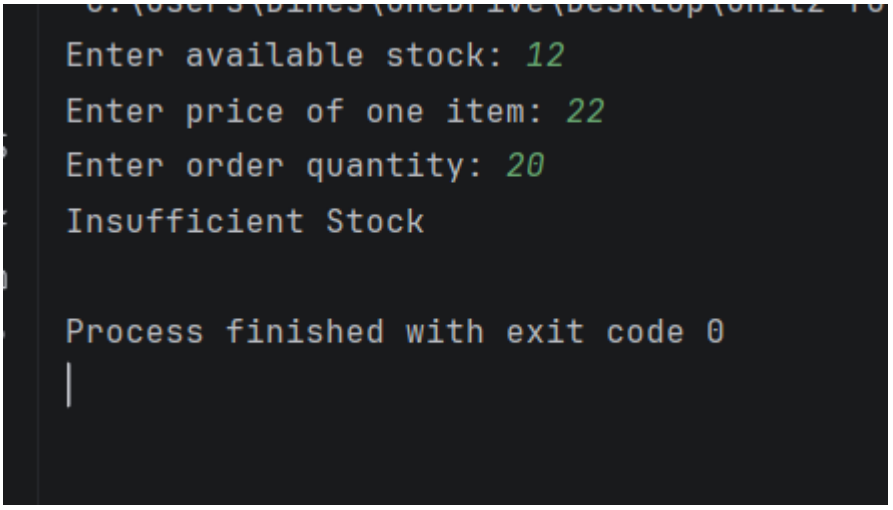
Enter available stock: 5

Enter price of one item: 20

Enter order quantity: 8

Insufficient Stock

Output Screenshots:



Conclusion:

The Smart Canteen Order and Stock Management program checks stock availability, calculates the customer's bill, and updates the remaining stock after a successful order.

Casestudy7:Library Borrowing and Fine Reconciliation: Analyse tuples representing book identifiers, borrowing periods, and return delays; calculate fines, identify overdue records, and reconcile book availability.

Problem Statement:

A college library allows students to borrow books for 14 days. If a student returns a book after 14 days, a fine of ₹5 is charged for each extra day. This program calculates the fine based on the number of days the book was kept**.**

Objectives:

1.Enter the number of days a book was borrowed.

2.Check whether the book was returned within 14 days.

3.Calculate the fine for late returns.

4.Display the total fine.

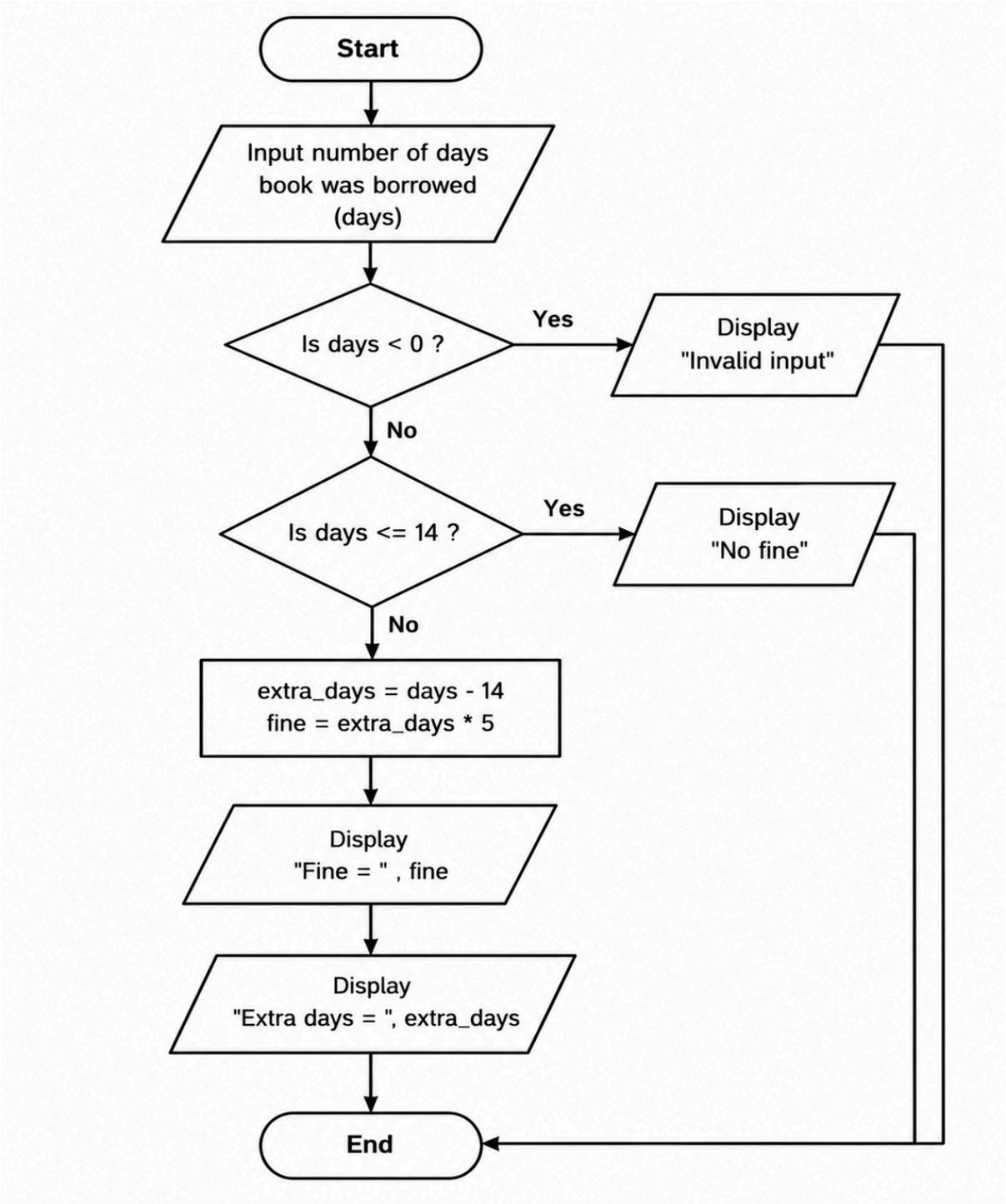
Assumptions:

- The allowed borrowing period is 14 days.
- The fine is ₹5 per extra day.
- The number of days cannot be negative.
- This simple program calculates the fine for one book.

Algorithm:

1. Start.
2. Enter the number of days the book was borrowed.
3. Check whether the number of days is negative.
4. If it is negative, display “Invalid input”.
5. If the days are 14 or less, display “No fine”.
6. Otherwise, calculate extra days and the fine.
7. Display the fine.
8. Stop.

Flowchart:



Pseudocode:

START

Input days

```
if  days < 0:
  Display  "Invalid input"
else if days <= 14:
    Display  "No fine"
else:
    extra_days = days - 14
    fine = extra_days * 5
   Display  "Fine:", fine
   Display  "Extra days:", extra_days
STOP
```

Python source code:

```
days = int(input("Enter number of days: "))
if days < 0:
    print("Invalid input")
elif days <= 14:
    print("No fine")
else:
    extra_days = days - 14
    fine = extra_days * 5
```

```
    print("Fine:", fine)
    print("Extra days:", extra_days)
```

Testcases:

#1

Enter number of days: 10

No fine

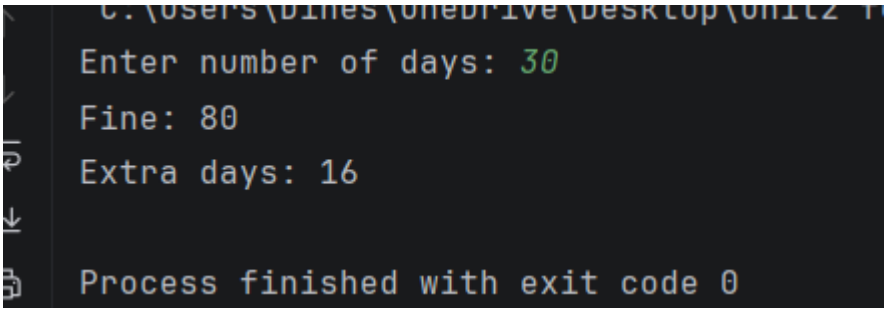
#2

Enter number of days: 20

Fine: 30

Extra days: 6

Output Screenshots:



Conclusion:

The Library Borrowing and Fine Reconciliation program checks the number of days a book was borrowed. It calculates the fine for late returns at ₹5 per extra day and displays the result. This helps a library calculate overdue fines easily.

Casestudy8:Student Performance Trend Analysis: Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

Problem Statement:

A college wants to compare a student's marks in two examinations. The program takes the marks from the first and second examinations and checks whether the student's performance has improved, decreased, or remained the same.

Objectives:

1. Enter marks from two examinations.
2. Check whether the marks are valid.
3. Compare the marks.
4. Display whether performance improved, decreased, or stayed the same.

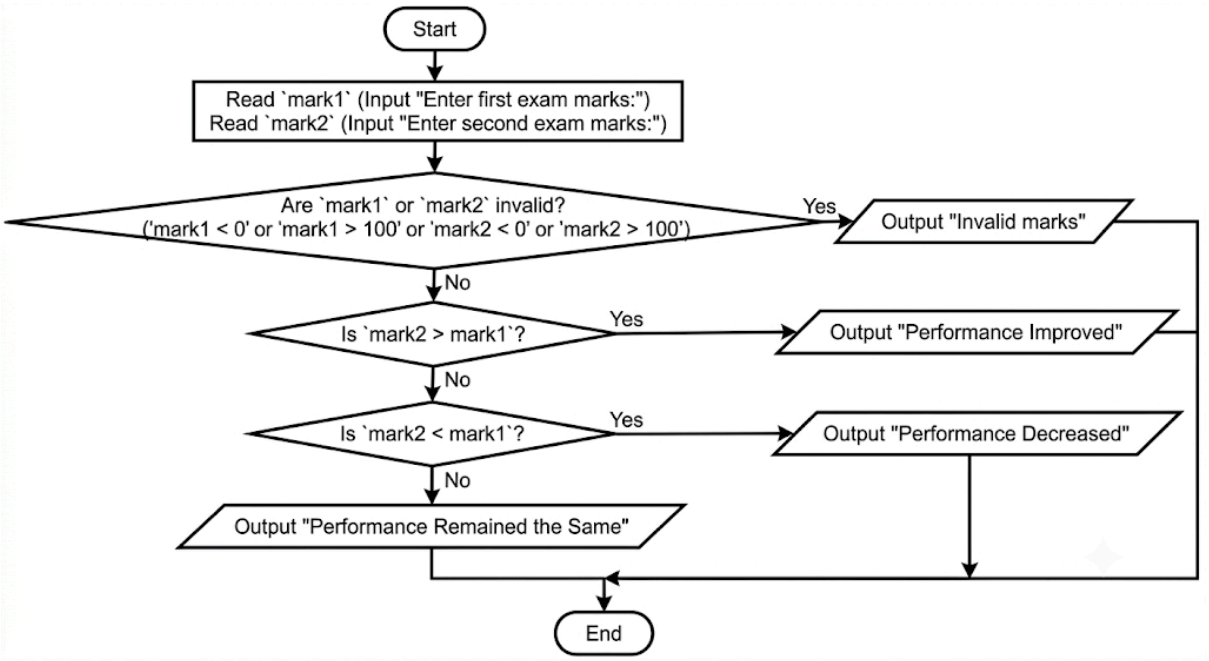
Assumptions:

- Marks are out of 100.
- Marks must be between 0 and 100.
- Only two examinations are considered.
- Higher marks in the second examination indicate improvement.

Algorithm:

1. Start.
2. Enter first examination marks.
3. Enter second examination marks.
4. Check whether both marks are between 0 and 100.
5. If marks are invalid, display an error message.
6. If second marks are greater, display “Performance Improved”.
7. If second marks are lower, display “Performance Decreased”.
8. Otherwise, display “Performance Remained the Same”.
9. Stop

Flowchart:



Pseudocode:

START

Input mark1

Input mark2

if mark1 < 0 or mark1 > 100 or mark2 < 0 or mark2 > 100:

    Display "Invalid marks"

else if mark2 > mark1:

  Display "Performance Improved"

else if mark2 < mark1:

  Display "Performance Decreased"

else:

  Display "Performance Remained the Same"

STOP

Python source code:

```
mark1 = int(input("Enter first exam marks: "))
mark2 = int(input("Enter second exam marks: "))
if mark1 < 0 or mark1 > 100 or mark2 < 0 or mark2 > 100:
    print("Invalid marks")
elif mark2 > mark1:
    print("Performance Improved")
elif mark2 < mark1:
    print("Performance Decreased")
else:
    print("Performance Remained the Same")
```

Testcases:

#1

Enter first exam marks: 65

Enter second exam marks: 80

Performance Improved

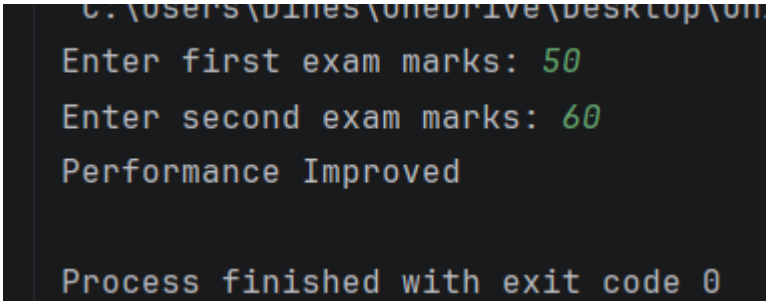
#2

Enter first exam marks: 85

Enter second exam marks: 70

Performance Decreased

Output Screenshots:



Conclusion:

The Student Performance Trend Analysis program compares marks from two examinations and identifies whether a student's performance has improved, decreased, or remained the same. It also checks for invalid marks.

Casestudy9:Hackathon Team Formation and Ranking: Analyse student scores and team compositions, validate

team sizes, aggregate round-wise scores, and resolve tied rankings using predefined rules.

Problem Statement:

A college organizes a hackathon in which teams participate and receive scores. This program accepts the scores of two teams, compares them, and displays which team ranks first. If both teams have the same score, it displays that the teams are tied.

Objectives:

1. Enter the scores of two teams.
2. Check whether the scores are valid.
3. Compare the scores.
4. Identify the team with the higher score.
5. Display a tie when both scores are equal.

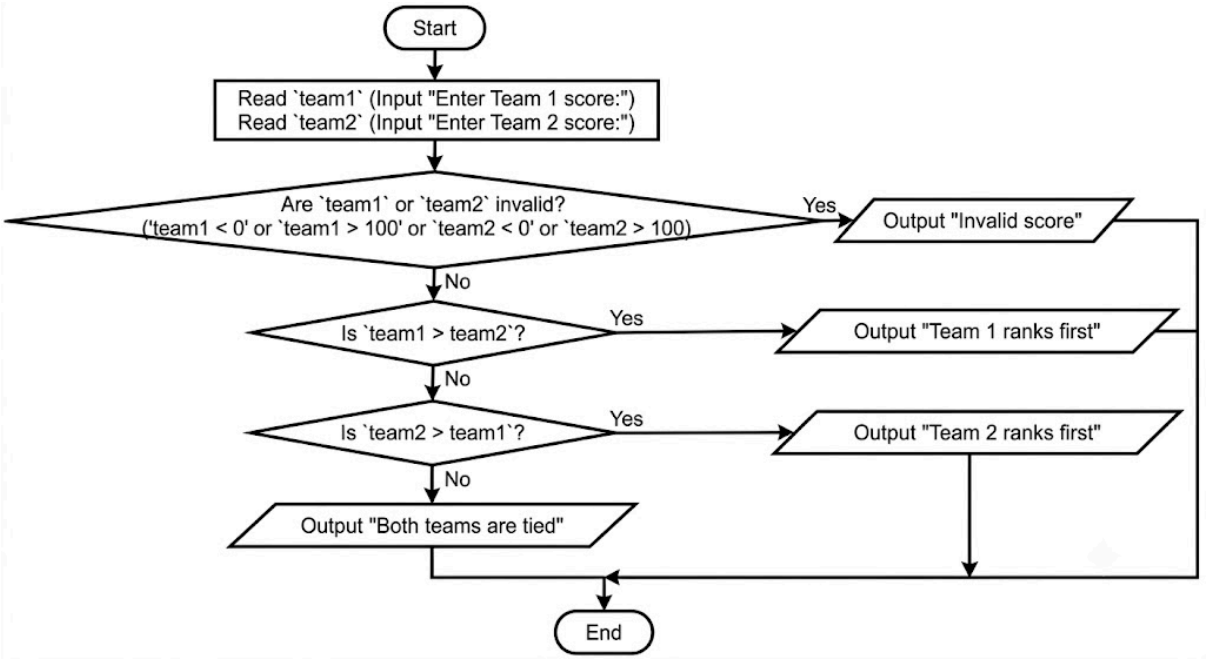
Assumptions:

- Scores are between 0 and 100.
- Only two teams are considered in this simple version.
- A higher score means a better rank.
- Both teams receive a score.

Algorithm:

1. Start.
2. Enter Team 1's score.
3. Enter Team 2's score.
4. Check whether the scores are between 0 and 100.
5. If the scores are invalid, display an error message.
6. If Team 1's score is higher, display Team 1 as the winner.
7. If Team 2's score is higher, display Team 2 as the winner.
8. Otherwise, display that both teams are tied.
9. Stop.

Flowchart:



Pseudocode:

```
START
Input  team1
Input  team2
if  team1 < 0 or team1 > 100 or  team2 < 0 or  team2 > 100:
    Display  "Invalid score"
elseif  team1 > team2:
    Display  "Team 1 ranks first"
else if   team2 > team1:
  Display "Team 2 ranks first"
else:
   Display  "Both teams are tied"
STOP
```

Python source code:

```
team1 = int(input("Enter Team 1 score: "))
team2 = int(input("Enter Team 2 score: "))
if team1 < 0 or team1 > 100 or team2 < 0 or team2 > 100:
    print("Invalid score")
```

```
elif team1 > team2:
    print("Team 1 ranks first")
elif team2 > team1:
    print("Team 2 ranks first")
else:
    print("Both teams are tied")
```

Testcases:

#1

Enter Team 1 score: 90

Enter Team 2 score: 75

Team 1 ranks first

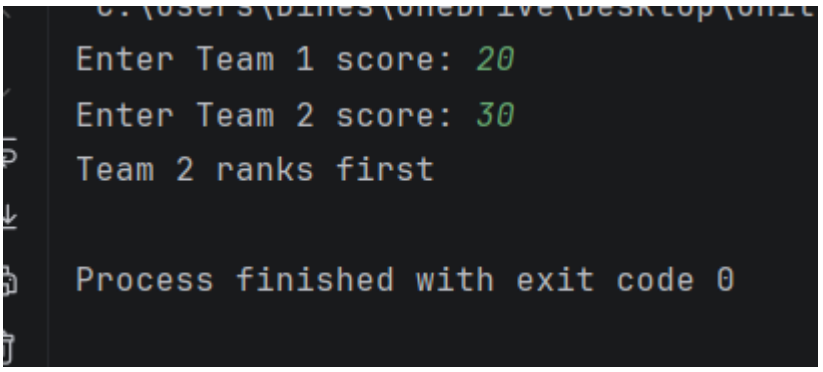
#2

Enter Team 1 score: 80

Enter Team 2 score: 80

Both teams are tied

Output Screenshots:



Conclusion:

The Hackathon Team Formation and Ranking program compares two team scores and identifies the team with the highest score. It also handles invalid scores and tied results.

Casestudy10:Integrated Student Record Consistency Checker: Validate student identifiers, names, marks, attendance, and course records stored in lists and tuples; detect duplicates, missing values, and inconsistent academic records.

Problem Statement:

A college maintains student records containing a student's name, roll number, and marks. Sometimes, the marks entered in two records may be different. This program compares the marks in two records and checks whether they match.

Objectives:

1. Enter the student's roll number.
2. Enter marks from two records.
3. Check whether the marks are valid.
4. Compare both records.
5. Display whether the records match or differ.

Assumptions:

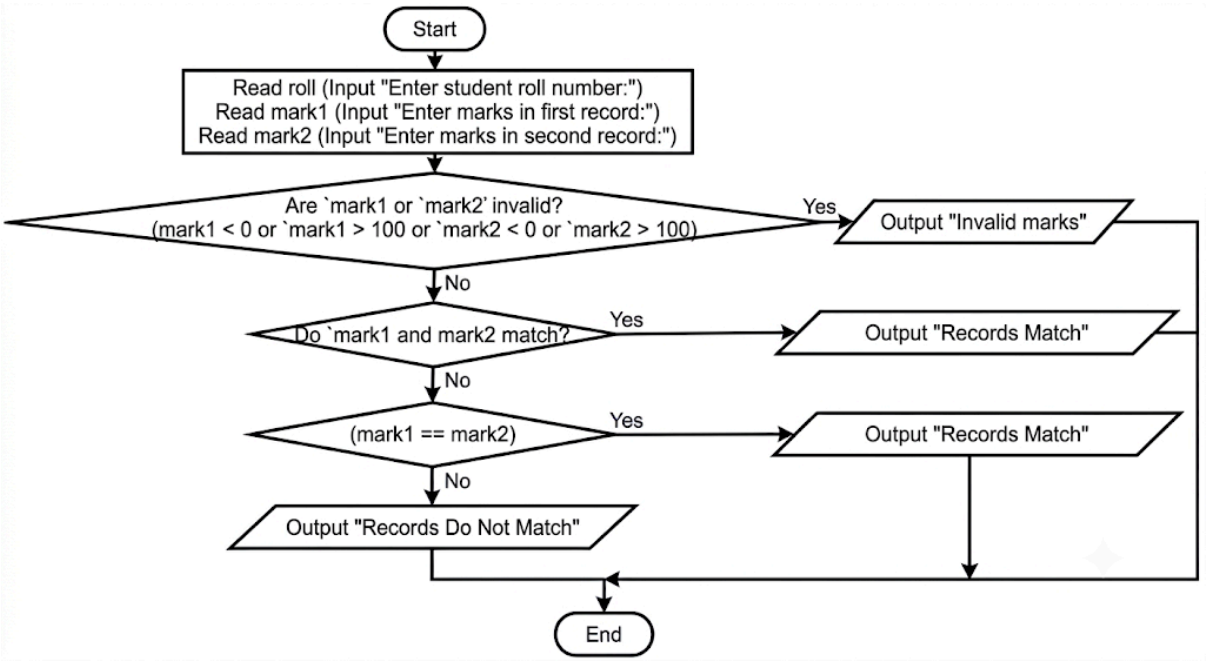
- Marks are out of 100.
- Marks must be between 0 and 100.
- The program compares records for one student.
- Both records belong to the same student.

Algorithm:

1. Start.
2. Enter the student's roll number.
3. Enter marks from the first record.
4. Enter marks from the second record.
5. Check whether the marks are valid.
6. If marks are equal, display “Records Match”.

7. Otherwise, display “Records Do Not Match”.
8. Stop.

Flowchart:



Pseudocode:

START

Input roll

Input mark1

Input mark2

if mark1 < 0 or mark1 > 100 or mark2 < 0 or mark2 > 100:

    Display "Invalid marks"

```
else if  mark1 == mark2:
    Display  "Records Match"
else:
   Display  "Records Do Not Match"
STOP
```

Python source code:

```
roll = input("Enter student roll number: ")
mark1 = int(input("Enter marks in first record: "))
mark2 = int(input("Enter marks in second record: "))
if mark1 < 0 or mark1 > 100 or mark2 < 0 or mark2 > 100:
    print("Invalid marks")
elif mark1 == mark2:
    print("Records Match")
else:
    print("Records Do Not Match")
```

Testcases:

#1

Enter student roll number: 102

Enter marks in first record: 75

Enter marks in second record: 80

Records Do Not Match

#2

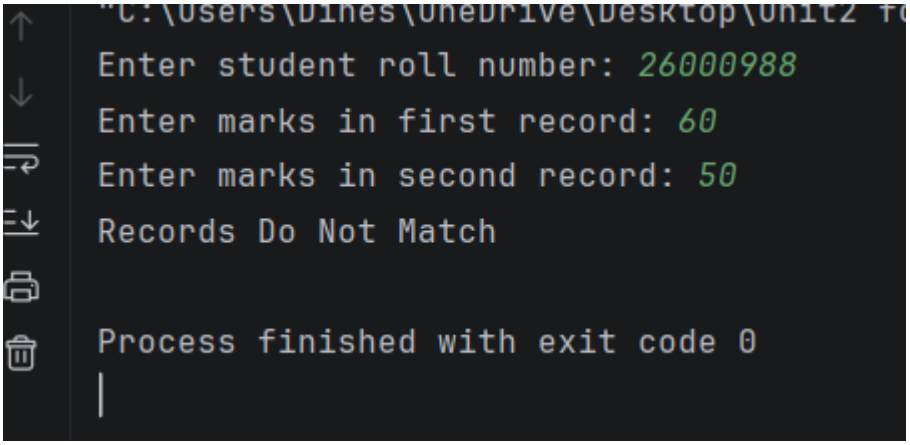
Enter student roll number: 101

Enter marks in first record: 85

Enter marks in second record: 85

Records Match

Output Screenshots:



Conclusion:

The Integrated Student Record Consistency Checker compares marks from two records for a student. It identifies matching and mismatching marks and helps detect data-entry errors.